

11. Node & Express

Attention!

This training is **interview-driven**.

To become a qualified developer, *ChatGPT* and *YouTube* will always be your best friends.

Outline

- [Node.js](#)
- Node.js Architecture
- Express
- MVC

Node.js

- An open-source, **single-threaded**, **backend JS runtime environment** that runs on the **V8** engine
 - JS Runtime Environment (Not a language or a framework)
 - Build on V8 Javascript engine
 - Written in C++
 - Single-threaded, event-driven, asynchronous(non-blocking) I/O
- **Why use node?**
 - Fast, efficient and highly scalable
 - Large ecosystem (npm)
 - Same language on the front and back end (JS)

Node.js vs Javascript

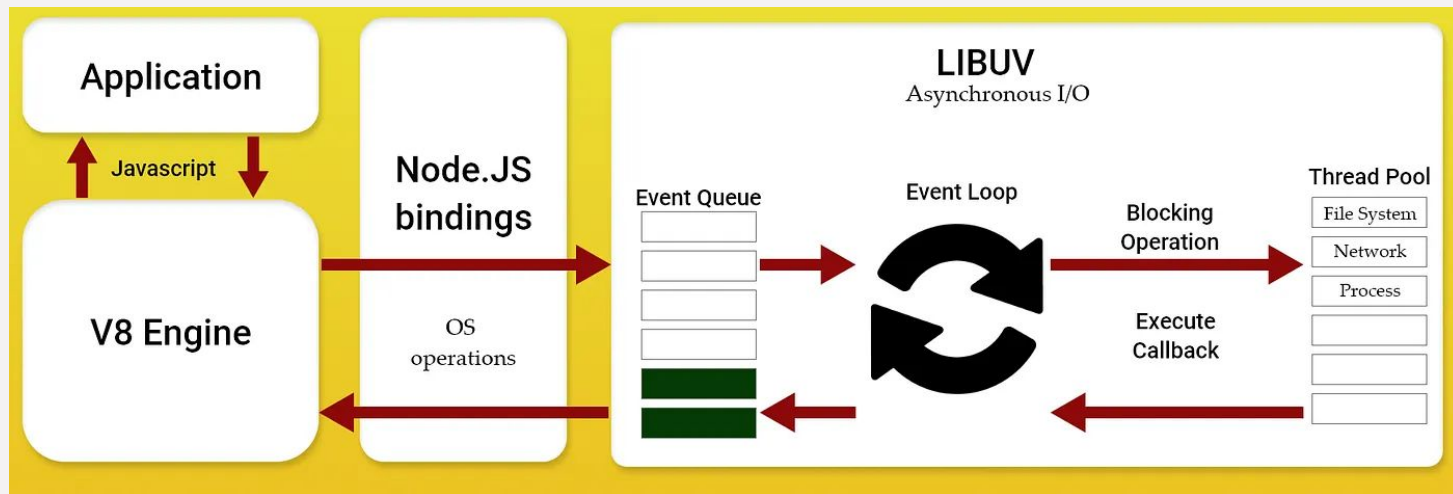
Feature	JavaScript (Browser)	Node.js
What it is	A programming language	A runtime environment for JS
Where it runs	In the browser	On the server / locally
Use case	Frontend logic	Backend apps, APIs, scripts
Common with	React, HTML, CSS	Express, MongoDB, REST APIs
Engine	V8 (Chrome), SpiderMonkey (Firefox), etc.	V8 (Chrome)

Node.js Architecture

- Node.js is not just a JavaScript framework; it is a runtime environment. It relies on two critical dependencies to function:
 - **V8 Engine:** Developed by Google (used in Chrome), it compiles JavaScript code into native machine code
 - **Libuv:** A C++ library that provides the **Event Loop** and handles asynchronous I/O operations
- Key Components
 - The Event Loop: Managing and executing asynchronous operations
 - The Call Stack (Single Thread)
 - Node APIs
 - The Callback Queue (Task Queue)

Node.js Architecture

Your JS code (single-threaded)
↓
V8 engine → compiles + executes JS
↓
libuv → handles non-blocking I/O via thread pool + event loop
↓
OS (files, network, DB)



Node.js event loop

- The event loop is the core of Node.js asynchronous, non-blocking architecture
 - Handle many operations (I/O, timers, callbacks) efficiently on a single thread
 - Process thousands of requests without blocking the thread
- The event loop runs in cycles. In each tick, it moves through these phases ->

```
└─ timers (setTimeout, setInterval)
   |
   └─ pending callbacks
      |
      └─ idle, prepare
         |
         └─ poll ← 处理 I/O (读文件、网络等)
            |
            └─ check ← setImmediate
               |
               └─ close callbacks
```


Node.js event loop

- Microtask Queues
 - **process.nextTick(callback)**
 - Always runs before any other timers at the end of the current phase
 - **Promise .then, .catch, .finally** callbacks
- Macrotask Queues
 - timers: this phase executes callbacks scheduled by **setTimeout()** and **setInterval()**
 - pending callbacks: executes I/O callbacks deferred to the next loop iteration
 - idle, prepare: only used internally
 - poll: waits for I/O or processes I/O if complete
 - check: **setImmediate()** callbacks are invoked here
 - close callbacks: some close callbacks, e.g. `socket.on('close', ...)`

Timers

- A module that allow you to schedule code execution
 - After a certain delay
 - At regular intervals
 - On the next cycle of the event loop

Function	What it does
<code>setTimeout</code>	Runs once after delay
<code>setInterval</code>	Runs repeatedly at interval
<code>setImmediate</code>	Runs on next event loop phase
<code>process.nextTick</code>	Runs before next tick (highest priority)
<code>clearTimeout</code>	Cancels a timeout
<code>clearInterval</code>	Cancels an interval
<code>clearImmediate</code>	Cancels an immediate

```
const fs = require('fs');

fs.readFile(__filename, () => {
  setTimeout(() => {
    console.log('timeout');
  }, 0);

  setImmediate(() => {
    console.log('immediate');
  });
});
```

NPM vs NPX

```
node -v
```

```
npm -v
```

- Use third-party libraries/packages in Node.js
- Node package manager (NPM)
 - a JS package manager that provides a command-line interface (CLI) for **installing and managing packages** from the npm registry
 - NPM is the default package manager for Node.js
 - NPM is one of the largest ecosystem of open source libraries in the world
 - Included when you install Node.js, needs install first, then run
- Node package execute (NPX)
 - A command-line tool that allows you to run Node.js binaries/packages without globally installing them
 - Use to run commands/binaries without installing
 - Useful for running a package just once or temporarily

NPM vs NPX

Feature	npm	npx
Purpose	Install/manage packages	Run package binaries (commands)
Installs package?	Yes (adds to node_modules)	No (downloads + runs temporarily, unless already installed)
Global install required?	Yes if you want to run CLI globally	No — runs package without installing globally
Example use case	Add lodash to your project	Run a tool like <code>create-react-app</code> or <code>eslint</code> just once

Node Version Management

- Node version management
 - Install multiple Node versions
 - Switch between versions easily
 - Set default versions
- Node Version Manager (NVM)
 - a command-line tool that lets you **install**, **manage**, and **switch** Node.js versions per shell/session
- n
 - implemented as a Node CLI tool
 - Installs and switches Node versions globally

✓ Install specific Node.js versions:

bash

```
nvm install 14.21.3
```

```
nvm install 20.5.1
```

✓ Switch between versions:

bash

```
nvm use 14.21.3
```

```
nvm use 20.5.1
```

✓ Set a default version (used in new shells):

bash

```
nvm alias default 20.5.1
```

✓ List installed versions:

bash

```
nvm ls
```

Package.json

- A metadata file for your Node.js project
 - Tracks your dependencies and scripts
 - Allows others to install your project easily
 - npm init / npm init -y
- name/version/description
- main: Entry point file
- scripts: Command shortcuts you can run with
npm run <script-name>
 - npm start => node index.js

```
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "A sample Node.js app",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "test": "echo \"No tests yet\""
  },
  "author": "Your Name",
  "license": "MIT",
  "dependencies": {
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^2.0.22"
  }
}
```

Package.json

- dependencies: Packages needed at **runtime**
 - npm install: checks package.json and locally installs all dependencies
 - npm i packageName:
 - locally installs a package, automatically adding to package.json
 - npm i -g packageName:
 - Installs the package globally, no need to install in each project
- devDependencies: Packages needed only during **development**
 - Won't be installed in production when you deploy
 - npm i --save-dev **Nodemon** (automatically restarts a node application)
- node_modules: the folder containing all locally-installed packages, enabling to import
 - Always regenerated when using “npm install”
 - Never include them in zipped files or when uploading to code-hosting services

Package.json

- **dependencies:** needed at runtime (app won't work without them)
- **devDependencies:** only needed during development (testing, building, etc.)
- **npm i:** installs everything listed in `package.json`
- **npm i packageName:** installs + adds to `dependencies`
- **npm i -g packageName:** installs globally, doesn't affect `package.json`

Modules - Module systems

- 2 main module systems
 - CommonJS (CJS): The original Node.js module system
 - Uses `require` and `module.exports`
 - ES Modules (ESM): The standard in modern JS (browser + Node.js)
 - Uses `import` and `export`

Feature	CommonJS (CJS)	ES Modules (ESM)
Import syntax	<code>require()</code>	<code>import</code>
Export syntax	<code>module.exports</code> / <code>exports</code>	<code>export</code> , <code>export default</code>
Loading	Synchronous	Asynchronous
File extensions	<code>.js</code> (default)	<code>.mjs</code> or <code>.js</code> with <code>"type": "module"</code>
Browser support	No (needs bundler)	Yes (modern browsers)

Global variable

- Built-in objects/functions that are available everywhere in your Node.js app
 - No need to `require()` or import anything
 - `process.env..PORT/NODE_ENV/DB_URL/JWT_SECRET`

Global	What it is	Example use
<code>__dirname</code>	Dir path of current module	Build file paths
<code>__filename</code>	File path of current module	Debug, logging
<code>exports</code>	Shortcut for <code>module.exports</code>	Export pieces of your code
<code>module</code>	Metadata + <code>exports</code> object	Full control of exports
<code>require()</code>	Import other modules	Load and use modules
<code>process</code>	Node process info/control	Env vars, exit, args

Module Types

- Node.js modules help you keep code clean and organized
 - **Core** Modules: built-in modules — no need for npm install
 - **Local** Modules: CommonJS/ES Modules
 - **Third-party** modules: Packages installed via NPM

Type	Source	How to use	Example
Core Module	Built into Node	<code>require('fs')</code>	<code>fs</code> , <code>http</code> , <code>path</code>
Local Module	Your code files	<code>require('./myModule')</code>	<code>./utils.js</code>
Third-Party Module	npm packages	<code>require('express')</code>	<code>express</code> , <code>lodash</code>

Module Types - Core Modules

- **Core Modules:** built-in tools in Node.js — no install needed

Module	Purpose	Example use
<code>Buffer</code>	Handle binary data	Streams, file I/O
<code>http</code>	Create servers/clients	APIs, web servers
<code>fs</code>	Work with files	Read/write files
<code>path</code>	Build file paths	Safe file path operations
<code>url</code>	Parse/build URLs	Work with request URLs
<code>child_process</code>	Run external commands	Execute shell tools
<code>os</code>	Get OS info	Monitor system resources

events

- events: It provides the **EventEmitter** class, which allows objects to emit named events and **register listeners** for those events
 - `on(event, listener)`: Add a listener (can run multiple times)
 - `once(event, listener)`: Add a listener that runs only once
 - `emit(event, [...args])`: Fire the event, optionally pass data
- EventEmitter is built on the Pub-Sub pattern
 - publish-subscribe pattern
 - Publisher: Emits event
 - Subscriber: Listens + reacts to event
 - Key idea:
 - Publishers don't know who (or how many) subscribers there are
 - Subscribers don't need to know where the event came from
 - This decouples who triggers the event from who responds to it

http: Building a Web Server

- “http” provides `createServer()` method to create a web server.
 - `http.createServer`
 - Creates a basic HTTP server
 - **req**: Contains incoming request info
 - **res**: Used to send response back
 - `res.writeHead(statusCode, headers)`
 - `res.end([data])`: signal that the response is complete
 - **server.listen(port)**: Starts server listening on port

Express

- A minimal and flexible web framework for Node.js.
- Express simplifies:
 - Routing (handling different URLs and HTTP methods)
 - `if (req.url === '/') => app.get('/')`
 - Request & Response
 - `req`: request data (params, query, body, headers)
 - `res`: response helpers (send, json, status)
 - Middleware handling (e.g., parsing request bodies, logging, authentication)
 - `app.use(express.json())`
 - Error handling
 - Serving static files (like HTML, CSS, images)

Express

```
const http = require('http');

const server = http.createServer((req, res) => {
  // Manual Routing: You must use if-else to match URLs
  if (req.url === '/' && req.method === 'GET') {
    // Manual Headers: You must set status 200 and content-type
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Home Page');
  } else if (req.url === '/users' && req.method === 'POST') {
    // Handling data (like JSON) is very complex in raw Node
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ message: 'User Created' }));
  } else {
    // Manual 404 handling
    res.writeHead(404);
    res.end('Page Not Found');
  }
});

server.listen(3000);
```



```
const express = require('express');
const app = express();

// Automatic Routing: .get() matches the method and path for you
app.get('/', (req, res) => {
  // Smart Response: .send() automatically sets status 200 and headers
  res.send('Home Page');
});

app.post('/users', (req, res) => {
  // .json() automatically converts objects and sets JSON headers
  res.json({ message: 'User Created' });
});

// This handles all unmatched routes (404) automatically
app.use((req, res) => {
  res.status(404).send('Page Not Found');
});

app.listen(3000);
```


Request object (req)

- Request (req): Extract data sent by the user, Client (Browser) -> Server
 - req.body: Used in **POST** or **PUT** requests
 - res.send(req.body.username)
 - req.params (URL Variables): Used for **dynamic** routes
 - Route: /users/:id
 - req.query (Search Parameters): Used for data after the **?** in the URL
 - Commonly used with **GET** requests
 - Used for data after the **?** in the URL
 - /users?page=2&limit=3
 - Usage: filtering, sorting, pagination, search
 - Not in the route: They don't affect the route path itself

Route params vs Query params

Feature	Route Parameter	Query Parameter
Syntax	<code>/users/:id</code>	<code>/users?page=2&limit=10</code>
Accessed with	<code>req.params.id</code>	<code>req.query.page</code> , <code>req.query.limit</code>
Appears where	In the path	After a <code>?</code> in the URL
Typical usage	Identify a resource	Control/filter how data is returned
Optional?	❌ Usually required	✅ Optional
Multiple values	❌ One per placeholder	✅ Can have many query fields

Response object (res)

- Response (res): HTTP response, Server -> Client (Browser)
 - res.send(): Sends text or HTML as a response
 - res.json(): Sends a JSON object as a response
 - res.status(code): Setting Codes
 - res.status(404).send('Not Found')
 - res.redirect(url): Redirects the client to a different URL
 - res.sendFile(path): Sends a file as a response
 - res.end([data]): terminate the response, close connection with the client

Port

- A Port is the virtual "address" your server uses to listen for incoming requests, ranging from 0 to 65,535
- `app.listen(port, cb)`: binds this host & port and waits for incoming requests
 - `const PORT = process.env.PORT || 3000;`

Port	Purpose
80	HTTP (default web traffic)
443	HTTPS (secure web traffic)
3000	Common default for local development
5000	Also popular for dev APIs
8080	Alternate HTTP port

Routing

- A route defines how an application processes incoming requests
 - `app.METHOD(PATH, HANDLER)`
 - **HTTP method:** Specify the type of request
 - **Path (URL):** Requested resource
 - **Route handler:** This function processes the request and sends a response
 - **`express.Router()`:** a function that creates modular route handlers
 - Split routes into multiple files
 - Organize routes by feature/module
 - Modularity, reusability, cleaner code

Routing

```
// ❌ server.js (Messy)
const app = express();

// User routes
app.get('/users', ...);
app.get('/users/profile', ...);
app.get('/users/settings', ...);

// Product routes
app.get('/products', ...);
app.get('/products/:id', ...);
// ... imagine 100 more lines of this
```

```
// file: routes/users.js
const express = require('express');
const router = express.Router(); // 1. Create the Router

// 2. Define routes on the router (Notice we don't write '/users' here)
router.get('/', (req, res) => {
  res.send('List of users');
});

router.get('/profile', (req, res) => {
  res.send('User Profile');
});

// 3. Export the router so the main app can use it
module.exports = router;
```

```
// file: server.js
const express = require('express');
const app = express();

const userRoutes = require('./routes/users'); // Import the mini-app

// MOUNTING: "Any request that starts with /users, send it to the userRoutes"
app.use('/users', userRoutes);

app.listen(3000);
```

Route defined in	Prefix in	Final URL User Visits
/	/users	localhost:3000/users
/profile	/users	localhost:3000/users/profile

Middleware

- Middleware is a function that gets executed between the request and the response
 - 3 parameters: req, res, **next** (to pass control to the next middleware)
 - Executes in the order they're applied
- next() function
 - It's a function to pass control to the next middleware
 - If next() is not called inside a middleware function, Express will not proceed to the next function, causing the request to hang indefinitely



Middleware

Purpose	Examples
 Authentication	Check if the user is logged in
 Logging	Log request info for debugging
 Body parsing	Parse JSON, form data, etc.
 CORS handling	Set headers to allow cross-origin
 Static file serving	Serve HTML, CSS, images
 Error handling	Catch and handle server errors

```
// [Definition] The Guard Logic
const authMiddleware = (req, res, next) => {
  const token = req.headers['authorization'];

  if (token === 'secret-password') {
    console.log('✅ Identity Verified');
    next(); // CRITICAL! Move to the next step.
  } else {
    console.log('❌ Identity Failed');
    res.status(403).send('Access Denied!'); // Intercept. The flow ends here.
  }
};

// [Usage] Protect specific routes
// Only if authMiddleware calls next() will the actual route handler run.
app.get('/profile', authMiddleware, (req, res) => {
  res.send('This is your private profile data');
});
```


Static vs Dynamic website

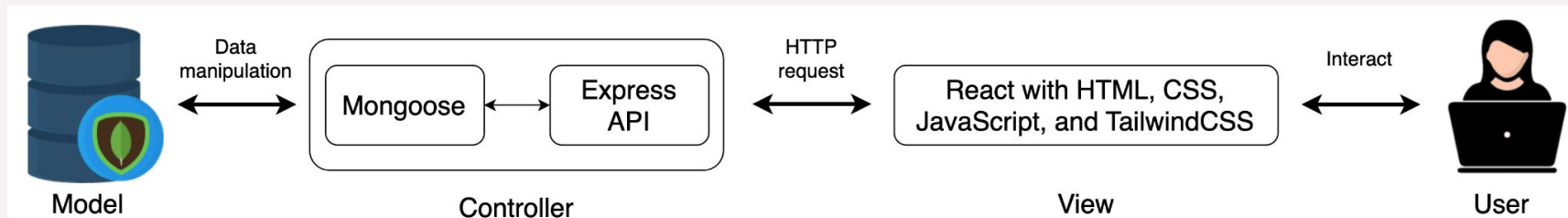
- Static website: pages are hardcoded and are displayed the same to all users (plain HTML, CSS, JS)
 - Pages always have the exact same contents & structure
 - HTML is not generated when navigating to a page
 - `express.static()`
- Dynamic website: pages may have different contents & structure for different users
 - HTML is programmatically generated when navigating to a page
 - **Server Side Rendering**

Static vs Dynamic website

Feature	Static Website	Dynamic Website
Content type	Fixed content (HTML/CSS/JS files)	Content generated dynamically using data/code
Server logic	No server-side logic	Uses server-side logic (e.g. Node.js, database)
Example	Portfolio, landing page	Blog, e-commerce site, dashboards
Performance	Fast, cached easily	Slower (depends on data processing)
Use case	Same for all users	Customized for each request/user

MVC architecture

- MVC = Model - View - Controller, a **design pattern** that helps organize code
 - Model: database
 - Handles data logic, database interactions, and business rules
 - Mongoose schemas/models, MongoDB collections
 - View: frontend
 - Represents the UI, React components
 - Controller: backend
 - Handles user input, processes requests, and returns responses
 - Express routes, controller functions



MVC architecture

```

└─ client
  └─ node_modules
  └─ public
  └─ src
    └─ components
    └─ pages
    └─ redux
    └─ styles
    └─ utils
    └─ App.tsx
    └─ index.tsx
    └─ react-app-env.d.ts
    └─ setupTests.ts
    └─ .gitignore
    └─ package-lock.json
    └─ package.json
    └─ README.md
    └─ tsconfig.json
```

```

└─ server
  └─ node_modules
  └─ src
    └─ config
    └─ controllers
    └─ middleware
    └─ models
    └─ public
    └─ routes
    └─ utils
    └─ index.ts
    └─ server.ts
    └─ .env
    └─ .gitignore
    └─ package-lock.json
    └─ package.json
    └─ tsconfig.json
    └─ .gitignore
    └─ package.json
```

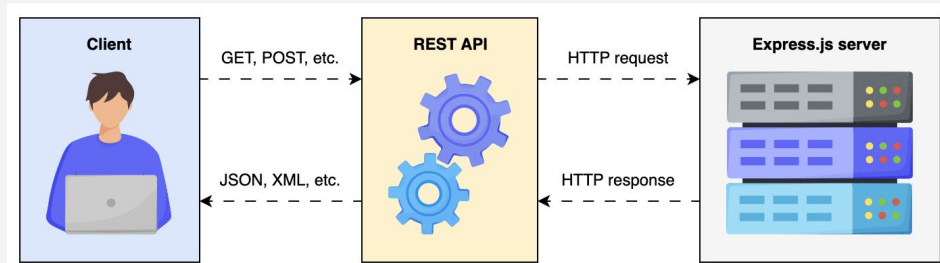
```

project/
├─ models/
│   └─ User.js
├─ controllers/
│   └─ userController.js
├─ routes/
│   └─ userRoutes.js
├─ app.js
├─ server.js
└─ frontend/ (React App)
```

API

- API stands for Application Programming Interface
 - An API is a set of rules that allows one program to talk to another
- In Express.js, an API is built using routes that respond to HTTP requests like:

- GET (fetch data)
- POST (create data)
- PUT (update data)
- DELETE (delete data)



- **REST API** (Representational State Transfer) Design Principles:
 - Resources are identified by URLs (/todos, /users)
 - HTTP methods define the action (CRUD)
 - Stateless: Each request must contain all necessary information
 - Responses use standard HTTP status codes (e.g. 200, 404, 500)

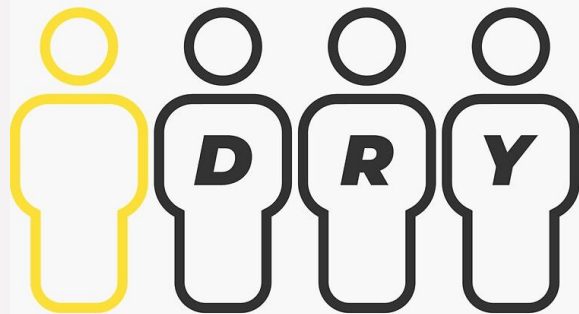
JSON

- JavaScript Object Notation, JSON is the standard format for data exchange in web apps
- `app.use(express.json())`: The tool you need to **read** incoming data
- `res.json()`: The tool you need to **send** outgoing data
 - Converts the Object into a JSON String
 - Sets the 'Content-Type' header to 'application/json'

Feature	JavaScript Object	JSON String
Where it lives	In your Node.js code (Memory)	In the Network Request/Response
Key format	<code>name: "Tom"</code> (Quotes optional)	<code>"name": "Tom"</code> (Quotes required)
Values	Can be functions, undefined, dates	Strings, Numbers, Booleans, Arrays, Objects only
Type	Object	String

DRY

- Don't Repeat Yourself (DRY): It's a software design principle that encourages **reusing** code instead of writing the same logic multiple times
 - Helps make Express apps clean, maintainable, and scalable
 - JS: functions, classes, modules
 - SCSS: partial, mixin, extend
 - EJS: partial, loops



Don't Repeat Yourself

CRUD

- CRUD is how you **Create, Read, Update, Delete** data
 - It stands for the 4 basic operations
- Express.js handle each operation using specific **HTTP methods** and **route** paths

Letter	Action	Description
C	Create	Add new data
R	Read	Get existing data
U	Update	Modify existing data
D	Delete	Remove data

Seed File

- A seed file is a standalone script that you run manually (or via an npm script) to populate your database with initial or fake data
 - (If database) Connect to the data store
 - Erase everything
 - Generate all the fake data
 - Save it to the store
- Usage: Run once to pre-fill your app's data
- Formats: JSON file, local array, or real database
- Usage: During setup, testing, demo, or resetting dev data
- Tools: Mongoose (MongoDB), [Faker.js](#), Sequelize, Prisma, etc.

Third-party library

- env(Environment Variables): Store environment variables like database URLs, API keys, and port numbers
 - Keeps sensitive data out of your code
 - Tool: [dotenv](#)
- uuid(Universally Unique ID): A library to generate unique identifiers
 - Tool: [uuid](#)
- Moment(Date/Time Formatting): A library to parse, format, and manipulate dates and times
 - Easier than using raw JavaScript Date
 - Tool: [moment](#), [dayjs](#)
- Lodash(Utility Functions): A utility library for working with arrays, objects, strings, etc
 - Tool: [lodash](#)

Postman

- A popular API client tool lets you send HTTP requests and view responses
 - Test, debug, and validate API endpoints

Additional materials

- Template Engine
- EJS

Homework - Short Answer Questions

1. What is Node.js?
2. What are some differences between Node.js and JavaScript?
3. What is npm? Difference between npm and npx?
4. What is CommonJS? How does it differ from ES Modules?
5. How do we import files into other files in Node.js?
6. How can you make the server automatically restart when files are modified?
7. What are some global objects in Node.js?
8. Explain how the Node.js architecture is event-driven in terms of event emitters and other core modules.
9. What are streams? In Node.js, what are the different kinds of streams?
10. What are the different timing or scheduling functions in Node.js?

Homework - Short Answer Questions

11. How does the event loop work in Node.js?
12. How do you manage multiple node versions?
13. Explain the request & response cycle. 1. What is Express.js?
14. How do you build a basic server in Node.js using express?
15. What is middleware?
16. What are ports? Which one is reserved for HTTP and HTTPS?
17. What is the difference between `response.send()` and `response.write()` in express?
18. Explain query strings vs route parameters.
19. How to use Express?
20. How to use middleware when using Express.js?

Homework - Short Answer Questions

21. What is the parameter next used for in Express?
22. What is the difference between fs module's readFile and createReadStream?
23. What are static websites?
24. What are dynamic websites?
25. What are the major differences between static and dynamic websites?
26. What are template engines and their advantages?
27. What is DRY?
28. What is CRUD?
29. What is a seed file for?
30. What is a REST API and how does it work?

Homework - Short Answer Questions

31. What are the key principles of RESTful APIs?
32. What is the difference between stateful and stateless APIs?
33. What is idempotency in REST APIs? Why is it important?
34. What are some common HTTP headers used in REST API requests and responses?
35. What is Postman, and why is it used?
36. What is the difference between query parameters and path parameters in a REST API?
37. When should you use query parameters instead of path parameters?
38. What happens if a required path parameter is missing in a request?
39. Can you use both path parameters and query parameters in the same API request? Give an example.
40. What is the proper way to handle optional query parameters in a REST API?

Homework - Short Answer Questions

- 41. What is JSON, and why is it widely used in REST APIs?
- 42. How does JSON handle data types like numbers, strings, array
- 43. What is JSON schema validation, and why is it important?

Homework - Coding Question

Test any one API from your full-stack project in Postman and take a screenshot, post it in Wechat group.